Objectifs RTO / RPO

> ***Statut : cible.*** *Ce document décrit ce que l'équipe vise et a décidé, pas ce qui est déjà codé. L'état réel du code est suivi dans* `constat/etat-implementation.md`*.*

> *Ticket : WOR-70 · Responsable : Cédric · Version 0.1 du 19/10/2026*
>
> *Ce document fixe les objectifs à partir du coût d'un arrêt (*`services-critiques.md`*) et des seuils de DORA (*`nis2-dora.md`*). Il définit aussi* ***comment les mesurer*** *pendant les essais (WOR-79), pour que la comparaison objectif / mesuré soit vérifiable. Il alimente le PCA/PRA (WOR-71).*

1. Définitions retenues

- **RTO** (durée maximale d'interruption) : temps entre le **début de l'impact** (première transaction en échec) et la **première transaction réussie et vérifiée** après la reprise. On compte depuis l'impact, pas depuis la détection : c'est ce que subissent les partenaires, et c'est ce que DORA mesure (« service downtime »).
- **RPO** (perte de données maximale admissible) : quantité de **transactions confirmées au client** qui manquent ou sont dupliquées après la reprise. On l'exprime en nombre de transactions (et en durée équivalente quand une restauration est en jeu).
- **Reprise vérifiée** : API accessible, transaction test réussie, données cohérentes (WOR-77). Un conteneur redémarré ne compte pas.

2. Deux niveaux d'objectifs

Niveau	À quoi il sert	D'où il vient
Objectif métier	Ce que l'entreprise fictive exige ; c'est ce qu'on présente à la direction	Coût d'un arrêt : au-delà de 5 min, l'incident sort de G1 ; au-delà de 30 min, l'incident passe en G3 : le budget SLA de 43 min par mois est presque consommé (pénalité d'environ 650 k€ au-delà) et la notification DORA devient probable
Cible du démonstrateur	Ce qu'on vise et mesure sur le PC Hôte	Objectif métier, avec de la marge, et la contrainte du Clip Chrono : 60 s en continu entre l'attaque et le service rétabli (WOR-83)

3. Objectifs par service

Service	Criticité	RTO métier	RPO métier	Cible démonstrateur	Mécanisme prévu
Traitement des transactions (API + BDD)	Vitale	**≤ 5 min**	**0** transaction confirmée perdue ou dupliquée	RTO ≤ 60 s (attaque A) · ≤ 3 min (attaque B)	Remédiation automatique (WOR-38), playbooks Ansible (WOR-44)
Authentification	Vitale	≤ 5 min	Sans objet : une session perdue oblige seulement à se reconnecter	Retour en même temps que l'API	Redémarrage ; après une rotation de secret, reconnexion des utilisateurs acceptée
WAF / point d'entrée	Vitale	≤ 5 min	Sans objet ; la configuration est versionnée	Coupure de quelques secondes à chaque changement de règle, à mesurer	Pas de rechargement à chaud : le WAF redémarre quand une règle change (`config_watch`)

Service	Criticité	RTO métier	RPO métier	Cible démonstrateur	Mécanisme prévu
Journalisation et SIEM	Importante	≤ 1 h	**0** journal perdu	Aucune perte pendant les essais	Export des journaux (`scripts/export-logs.sh`) **avant** toute restauration
Consultation de l'historique	Importante	≤ 30 min	0	Retour en même temps que l'API	Même service que l'API
Supervision, détection ML, remédiation	Support critique	≤ 15 min	Sans objet ; le modèle est versionné	Hors périmètre de l'attaque	Relance par Docker Compose
Assistant RAG	Non critique	Meilleur effort	Sans objet ; l'index peut être reconstruit	—	Réindexation

4. Répartition du RTO sur la chaîne de reprise

Le RTO se décompose en étapes. Chaque pilier tient une partie du budget. Les durées sont des **hypothèses à confirmer** avec Reza (collecte), Léa (détection) et Michael (remédiation).

Étape	Marqueur de temps	Responsable	Budget attaque A	Budget attaque B
Début de l'impact → anomalie visible dans les métriques	T1 → T2	Infrastructure (collecte Prometheus toutes les 5 s)	≤ 5 s	≤ 5 s
Anomalie visible → alerte ML	T2 → T3	IA / Data (fenêtre de 60 s évaluée toutes les 10 s, alerte après deux fenêtres anormales consécutives)	≈ 20 s	≈ 20 s
Alerte → décision de remédiation	T3 → T4	Développement	≤ 5 s	≤ 5 s
Décision → action terminée	T4 → T5	Infrastructure / Développement	≤ 15 s (redémarrage de l'API ; une limitation au WAF coûterait aussi un redémarrage du WAF)	≤ 2 min (blocage, restauration, rotation du secret)
Action terminée → transaction test réussie et données vérifiées	T5 → T6	Développement	≤ 10 s	≤ 20 s
Total (RTO mesuré = T6 – T1)			**≤ 60 s**	**≤ 3 min**

T0 désigne le lancement de l'attaque sur le PC Attaquant. L'écart T0 → T1 dépend de l'attaque et ne compte pas dans le RTO.

> ***Point de vigilance pour le Clip Chrono :*** *60 s, c'est T0 → T6* ***et*** *l'affichage du résultat. La collecte passe déjà toutes les 5 s (`infra/prometheus/prometheus.yml`). La détection reste le poste le plus long : environ 20 s si deux évaluations consécutives suffisent, jusqu'à 70 s si la fenêtre de 60 s doit se remplir d'abord (`docs/runbooks/reprise.md`). Le budget de 60 s est donc serré :* ***à mesurer en priorité avec Léa.***

5. Arbitrage à trancher : la restauration après une injection (attaque B)

WOR-29 prévoit un **retour à un snapshot propre de la base**. C'est incompatible avec un RPO de 0 : toutes les transactions légitimes validées entre le snapshot et la restauration seraient perdues.

Option	RPO obtenu	Effort	Avis
1. Snapshot périodique seul (par exemple toutes les 5 min)	Jusqu'à 5 min de transactions perdues	Faible	Simple, mais contredit l'objectif métier et provoque un incident majeur DORA (perte de données)
2. **Snapshot + rejeu des transactions légitimes** depuis un journal en ajout seul, en écartant celles qui viennent de la source bloquée	**0** si le rejeu est correct	Moyen	**Recommandé** : démontre l'objectif métier, et le tri entre légitime et malveillant est un bon argument pour le jury
3. Restauration à un instant précis (PITR, par exemple l'archivage des WAL avec PostgreSQL), juste avant la première requête malveillante	Proche de 0, mais les transactions légitimes postérieures sont perdues	Moyen à élevé	Solution industrielle de référence, à citer dans les recommandations
4. Correction ciblée : annuler les seules lignes altérées, sans restauration globale	0	Élevé et risqué	Non retenu pour un prototype de 4 jours

Proposition : option 2 pour la démo, option 3 citée comme cible industrielle. Dans tous les cas, **les journaux et l'état de la base compromise sont copiés avant la restauration** : ce sont des preuves pour l'investigation et la notification.

À valider avec Michael (WOR-38, WOR-39) et Reza (WOR-44) avant de figer WOR-29.

6. Protocole de mesure (pour WOR-79)

Temps

Chaque marqueur est horodaté automatiquement, sur une **même horloge** (le PC Hôte ; le PC Attaquant est synchronisé dessus ou son décalage est noté avant chaque essai).

Marqueur	Source
T0 lancement de l'attaque	Journal du script d'attaque (WOR-58)
T1 première transaction en échec	Générateur de trafic normal : première réponse en erreur ou en dépassement de délai
T2 anomalie visible	Prometheus : premier point hors norme
T3 alerte ML	Journal du service d'inférence (WOR-53)
T4, T5 décision, action terminée	Événements publiés par le moteur de remédiation (WOR-38)
T6 reprise vérifiée	Moteur de remédiation : transaction test et contrôle de cohérence réussis

Données

1. Le générateur de trafic enregistre chaque transaction **confirmée** (identifiant, montant).
2. Après la reprise, on compare cette liste au contenu de la base.
3. **Transactions manquantes + transactions en double = RPO mesuré.** L'objectif est 0.

Grille de résultats

À remplir pendant les répétitions (WOR-78) : trois essais minimum par attaque.

Essai	Attaque	T1 → T3 détection	T3 → T6 reprise	RTO mesuré	Transactions perdues ou dupliquées	Objectif tenu ?
1	A					
2	A					
3	A					
1	B					
2	B					
3	B					

On retient le **pire des trois essais**, pas la moyenne : un RTO se garantit, il ne s'espère pas.

7. Ce que le code permet de tenir au 20/10

- **RPO = 0 : pas tenable en l'état.** Dans le socle, l'API garde ses données en mémoire : un redémarrage, première réponse à l'attaque A, efface toutes les transactions. Il n'existe en plus aucune sauvegarde de la base. Minimum avant les essais : brancher l'API sur PostgreSQL et faire un `pg_dump` avant chaque essai.
- **Délai de détection : pas mesurable.** Le modèle de détection n'existe pas encore ; seule l'extraction des features (fenêtre de 60 s, pas de 10 s) est codée.
- **Vérification de la reprise (T6) : à construire.** `/health` répond toujours « ok » ; la preuve de reprise doit être une transaction de test réussie, pas ce contrôle.

Détail et sources : `constat/etat-implementation.md`.

8. Ce que ces objectifs ne couvrent pas

- **Perte du PC Hôte** (panne, rançongiciel R7) : pas de deuxième site, donc pas de RTO tenable. La reprise passe par une réinstallation depuis le dépôt Git (WOR-49), sans objectif chiffré. C'est un risque résiduel assumé, à porter dans le PCA/PRA (WOR-71).
- **Sauvegardes sur le même hôte** : le RPO de 0 ne tient que si la base n'est pas détruite avec son hôte.
- **Faux positif du modèle ML** : une remédiation déclenchée à tort provoque elle-même une coupure, que le RTO doit absorber. À mesurer pendant le chaos engineering (WOR-76).